Let's Reproduce GPT-2 (124M) (Karpathy): Synthesis

Nov 5, 2026 · @Giáo sư Thịnh

The big picture

The video takes the small GPT from the earlier videos and turns it into a faithful reproduction of OpenAI's GPT-2 124M, the smallest GPT-2, then trains it from scratch fast and cheaply on modern hardware. It is about four hours long, and most of it is about engineering, not new ideas. This doc is my synthesis from what I know of the video, not a transcript, so treat specific numbers as approximate.

Why it matters: GPT-2 124M is the smallest model that feels like a real language model, and it can be trained in roughly a couple of hours on one rented 8-GPU node. The video shows what separates a toy script from a real training run.

The path the video takes:

1. Rebuild the GPT-2 architecture in PyTorch, then load OpenAI's weights through Hugging Face to check that it matches.
2. Train from random initialization on a tiny dataset and confirm the model can overfit a batch.
3. Make training fast: lower precision, `torch.compile`, Flash Attention, and cleaner tensor sizes.
4. Copy the training recipe from the GPT-2 and GPT-3 papers: optimizer settings, learning rate schedule, batch size, weight decay.
5. Scale across GPUs with DistributedDataParallel and add gradient accumulation.
6. Switch to a real dataset (FineWeb-Edu), add validation and a HellaSwag evaluation, and run a full training.

The result: a model trained on about 10 billion tokens that reaches roughly the quality of the original GPT-2 124M on HellaSwag. The code lives in Karpathy's `build-nanogpt` repository, and the same design underlies nanoGPT.

Architecture and initialization

The GPT-2 124M configuration:

Setting	Value
`block_size` (context length)	1024
`vocab_size`	50,257 (256 bytes + 50,000 merges + 1 end-of-text token)
`n_layer`	12
`n_head`	12
`n_embd`	768
Parameters	about 124 million

How it differs from the small model of the earlier GPT video:

Aspect	Earlier video	GPT-2 reproduction
Tokens	Characters	GPT-2 BPE tokens through tiktoken
Activation	ReLU	GELU, using the tanh approximation (a historical choice kept so weights match OpenAI's)
Attention	Separate query, key, value layers per head	One combined linear layer for all heads, split into query, key, and value, with heads as a batch dimension
Embeddings	Separate input and output tables	Token embedding and output layer share one weight matrix
Normalization	Layer norm before attention and feed-forward	Same pre-norm layout, plus one final layer norm before the output layer

Weight tying: the token embedding table and the final linear layer that produces logits use the same matrix. This saves about 38 million parameters (768 × 50,257) and is a good prior: tokens that are similar in meaning should be similar on both the input and the output side.

Initialization:

- Linear layers and embeddings start from a normal distribution with standard deviation 0.02, and biases start at zero.

- Each block adds two things to the residual stream (attention and feed-forward), so the output projection of each is scaled down by `1 / sqrt(2 * n_layer)`. Without this the variance of the residual stream would grow with depth.

Checking the build:

1. Load the Hugging Face GPT-2 124M weights into your own model. Some tensors are stored transposed (they come from a `Conv1D` layer), so they need transposing, and the attention mask buffers are skipped.
2. Sample text with top-k sampling (k = 50) and confirm it reads like GPT-2's.
3. Then initialize from scratch. A correct start has a loss near `-ln(1/50257)`, about 10.8, because an untrained model should guess uniformly. If the first loss is far from that, something is wrong in the init or the code.
4. Overfit one small batch. The loss should fall almost to zero. If it does not, there is a bug.

Making training fast

The guiding idea: modern GPUs can do arithmetic much faster than they can move data from memory. Most speedups either shrink the data (lower precision), reduce trips to memory (fusing operations), or both. The video adds them one at a time, and measures tokens per second after each one, because step time alone changes when you change the batch size.

Technique	What you do	Why it helps
TF32	`torch.set_float32_matmul_precision('high')`	Matrix multiplies use tensor cores with fewer mantissa bits. Almost no accuracy loss, a large speedup on matmuls, but memory movement is still the limit

Technique	What you do	Why it helps
bfloat16 autocast	Wrap the forward pass in `torch.autocast(device_type='cuda', dtype=torch.bfloat16)`	Activations are half the size. Sensitive operations such as layer norm, softmax, and the loss stay in higher precision. Unlike fp16, bf16 keeps fp32's range, so no gradient scaler is needed
`torch.compile`	`model = torch.compile(model)`	Compiles the model and fuses many small operations into single GPU kernels, so data is read and written once instead of many times. Costs some startup time
Flash Attention	`F.scaled_dot_product_attention(q, k, v, is_causal=True)`	Never builds the full T × T attention matrix in memory, using a fused kernel with an online softmax. Does a bit more arithmetic but far less memory traffic. The compiler cannot discover this rewrite on its own

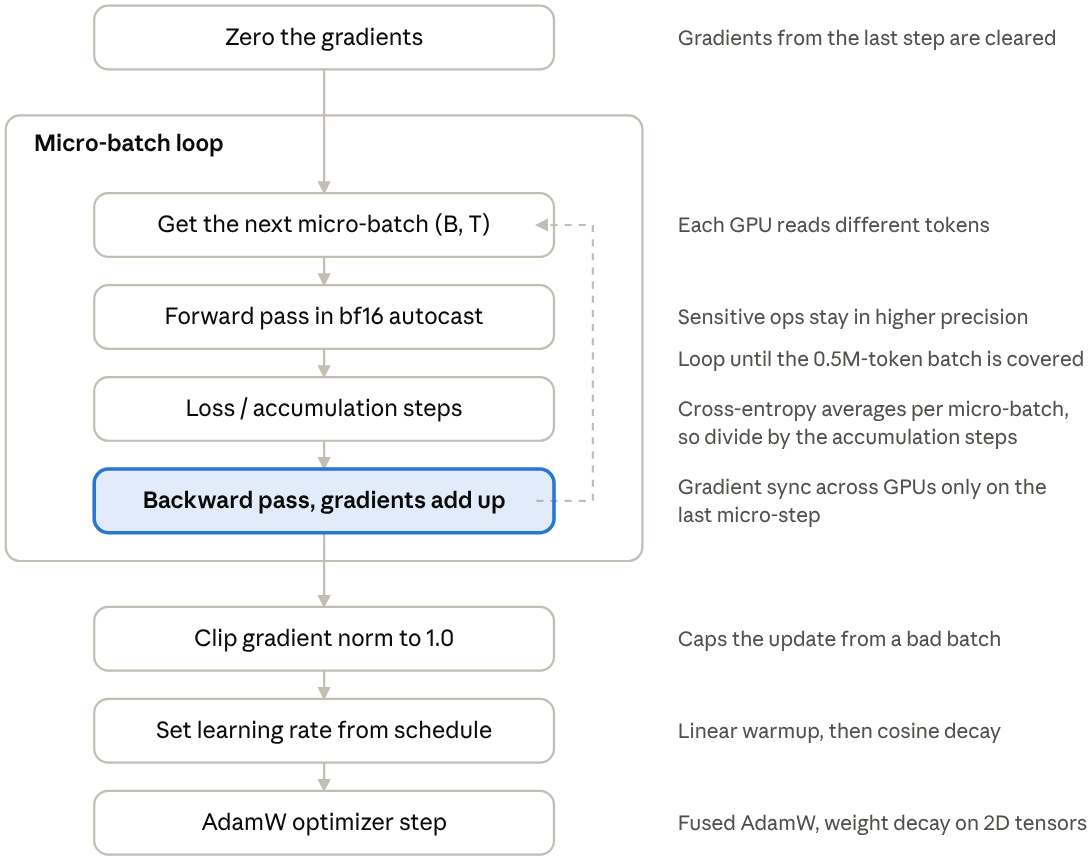
Technique	What you do	Why it helps
Nice numbers	Pad the vocabulary from 50,257 to 50,304	GPU kernels prefer sizes divisible by powers of two (here 128). The 47 extra tokens are never used, yet the run gets faster
Fused AdamW	`torch.optim.AdamW(..., fused=True)`	The optimizer update runs in fewer kernels instead of looping over each parameter tensor

Rule of thumb from the video: numbers that are not multiples of 8, 16, 64, or 128 are often slow on a GPU, so choose sizes accordingly. Padding to a bigger number can be faster than the smaller exact size.

What to measure: after each change print the step time and the tokens per second (`batch_size * block_size / time`). Drop the first few steps from your timing, because they include warmup and compilation.

One optimizer step at a glance

One optimizer step adds up many micro-batches before it updates



training step · 1 loop, 7 stages

The inner loop runs several micro-batches so a small GPU can imitate a 0.5-million-token batch. Only after the loop do the gradients get clipped, the learning rate get set, and the weights get updated. With several GPUs, each runs this same loop on different data, and their gradients are averaged on the last micro-step.

The optimization recipe

GPT-2's paper gives few training details, so the video borrows from the GPT-3 paper, which lists settings for a model of the same size (about 125M parameters).

Setting	Value	Why
Optimizer	AdamW, betas (0.9, 0.95), eps 1e-8	The GPT-3 settings

Setting	Value	Why
Gradient clipping	Global norm clipped to 1.0	A bad batch can produce a huge gradient and wreck training. Clipping caps the update. Printing the norm each step also shows instability
Learning rate	Linear warmup, then cosine decay from 6e-4 to 10% of that	Early steps are noisy, so start small. Decay lets the model settle at the end
Warmup length	About 375 million tokens, about 715 steps	From the GPT-3 paper
Weight decay	0.1, only on 2D tensors (matrix weights and embeddings)	Biases and layer norm parameters are not decayed
Batch size	About 0.5 million tokens (524,288 = 2^19) per optimizer step	The GPT-3 setting for this model size
Total training	About 10 billion tokens, about 19,000 steps, one pass over the data	Matches the data size of the FineWeb-Edu sample

Gradient accumulation: one GPU cannot hold a 0.5M-token batch at once. Instead, run several smaller micro-batches, add up their gradients, and take one optimizer step. One detail matters: cross-entropy averages the loss within each micro-batch, so divide each loss by the number of accumulation steps, or the gradient will be too large by that factor.

Why the batch size is counted in tokens: the batch of 524,288 tokens equals micro-batch size × sequence length (1024) × number of GPUs × accumulation steps. You keep the token count fixed and trade accumulation steps for GPUs.

Weight decay detail: build two parameter groups, one for tensors with two or more dimensions (decayed) and one for the rest (not decayed). Use the fused AdamW kernel when available.

Learning rate schedule in code: a function `get_lr(step)` returns a linear ramp for the warmup steps, the minimum rate after the final step, and a cosine curve in between. Set it on every optimizer param group before each step.

Distributed training on many GPUs

The idea (data parallelism): every GPU holds a full copy of the model and processes a different slice of the data. After the backward pass, the GPUs average their gradients, so all copies take the same optimizer step and stay identical.

How to run it: launch the same script once per GPU with `torchrun --standalone --nproc_per_node=8 train_gpt2.py`. `torchrun` sets environment variables (`RANK`, `LOCAL_RANK`, `WORLD_SIZE`) that tell each process which GPU it owns.

What changes in the code:

1. **Setup:** call `init_process_group(backend='nccl')`, and set `device = f'cuda:{ddp_local_rank}'` for each process. Let one process (rank 0) do the logging and saving, the "master process".
2. **Model wrapper:** wrap with `DDP(model, device_ids=[ddp_local_rank])`. Keep a reference to the raw model for saving and for the optimizer config.
3. **Different data per GPU:** the data loader starts each rank at a different offset and advances by `B * T * num_processes` tokens each step, so no two GPUs read the same tokens.
4. **Batch size math:** total batch = micro-batch × sequence length × number of GPUs × accumulation steps. With 8 GPUs there are fewer accumulation steps than with 1.
5. **Gradient sync only at the end of accumulation:** DDP normally averages gradients on every backward pass. During accumulation, set `model.require_backward_grad_sync = False` for all but the last micro-step, so the communication happens once per optimizer step.
6. **Average the logged loss:** each GPU sees different data, so average the accumulated loss across processes with `dist.all_reduce(loss_accum, op=dist.ReduceOp.AVG)` before printing.
7. **Clean up:** call `destroy_process_group()` at the end.

Hardware note: the video uses a cloud node with 8 A100 GPUs. Karpathy mentions renting such nodes by the hour from providers like Lambda, and that the full run costs on the order of tens of dollars. Prices change, so check the current rates.

Data: FineWeb-Edu

Early on the video trains on Tiny Shakespeare, only to debug and test speed. For the real run it uses **FineWeb-Edu**, a filtered collection of educational web pages from Hugging Face. The 10-billion-token sample is enough for one full pass for this model.

Preparing the data (a one-time script):

1. Download the 10B-token sample of FineWeb-Edu with the Hugging Face `datasets` library.
2. Tokenize every document with the GPT-2 tokenizer (tiktoken), prepending the end-of-text token to each document so the model sees where documents start.
3. Store tokens as `uint16` numbers in NumPy arrays, since the vocabulary fits in 16 bits. This halves the disk size versus 32-bit integers.
4. Write the result as shards of 100 million tokens each, saved as files on disk. The first shard serves as validation data, and the rest as training data.
5. Use multiprocessing to tokenize in parallel, because tokenizing 10B tokens on one core is slow.

The data loader at training time:

- Load one shard at a time and walk through it in order, taking `B * T + 1` tokens per step. The inputs are the first `B * T` tokens reshaped to (B, T), and the targets are the same tokens shifted by one position.
- When a shard runs out, move to the next one. With several GPUs, each process starts at a different offset and steps by `B * T * num_processes`.
- The documents are concatenated into one long stream, so a sequence can start in the middle of a document and contain several. The end-of-text token marks the boundaries, and the video notes this is a simple, slightly imperfect approach (no attention mask across documents).

Why data quality matters: the video points out that FineWeb-Edu is filtered for educational content, which helps a small model learn more per token than raw web text would.

Evaluation and results

Three signals to watch during the run:

1. **Validation loss:** every few hundred steps, average the loss over a handful of held-out batches in `eval` mode. If training loss falls but validation loss rises, the model is overfitting.
2. **Generated samples:** every few hundred steps, sample from a prompt such as "Hello, I'm a language model," and read the text. It is a quick sanity check that nothing broke.
3. **HellaSwag:** a common-sense benchmark where each question gives a short context and four possible endings, and only one is sensible.

How HellaSwag is scored here: small models do poorly with the multiple-choice format, so the video uses a completion-style comparison:

- Build four rows: the shared context followed by each ending.
- Run the model on all four and compute the average cross-entropy loss over just the ending tokens in each row.
- Pick the ending with the lowest loss. Two accuracies result: one using the summed loss, and a length-normalized one (`acc_norm`) that most published numbers use.

How the run compares (numbers approximate, from my memory of the video):

Model	Training data	HellaSwag (normalized)
Original GPT-2 124M	OpenAI's private WebText set (token count not clearly stated)	About 29%
The video's model	About 10 billion tokens of FineWeb-Edu	Slightly above GPT-2 124M
GPT-3 124M (from the GPT-3 paper)	About 300 billion tokens	About 34%

The main lesson: with cleaner data, better engineering, and a modern recipe, you can match the original GPT-2 124M with a modest training budget. Karpathy also notes that a longer run, with several times more tokens, gets closer to GPT-3's number. Validation losses are not directly comparable across different datasets, so compare models on a shared benchmark such as HellaSwag instead.

Run it yourself: the training script logs the validation loss, the HellaSwag accuracy, and the training loss to a file, and saves checkpoints. Plot them afterwards to see the curves.

Takeaways and a build checklist

Takeaways

1. The architecture is the small GPT you already know, with GELU, weight tying, a combined attention projection, and careful initialization.
2. Most of the gain comes from engineering: lower precision, fused kernels, Flash Attention, and GPU-friendly tensor sizes.
3. The training recipe matters as much as the model: AdamW settings, gradient clipping, warmup plus cosine decay, selective weight decay, and a large token batch via accumulation.
4. Data-parallel training across GPUs is a small code change on top of a single-GPU script.
5. Measure everything: tokens per second, gradient norm, validation loss, and a real benchmark such as HellaSwag.

Checklist to rebuild it yourself

- [ ] Write the GPT-2 model class and load Hugging Face weights to confirm it matches.
- [ ] Initialize from scratch, check the first loss is near 10.8, and overfit one batch.
- [ ] Add weight tying and the scaled residual initialization.
- [ ] Turn on TF32, bf16 autocast, `torch.compile`, Flash Attention, and the 50,304 vocabulary, measuring tokens per second after each.
- [ ] Add AdamW with the GPT-3 settings, gradient clipping, the learning rate schedule, and weight decay groups.
- [ ] Add gradient accumulation to reach a 524,288-token batch.
- [ ] Make the script run under `torchrun` with DDP, and sync gradients only on the last micro-step.
- [ ] Prepare FineWeb-Edu shards and write the sharded data loader.
- [ ] Add validation loss, sampling, and HellaSwag evaluation, then log to a file.
- [ ] Run the full training and plot the loss and HellaSwag curves.

Where to go next: the `build-nanogpt` repository (the code from this video), nanoGPT, and nanochat. If you want a coding model, continue with the roadmap: chat fine-tuning, then code data and tool use.